

Behavioural Design Pattern

14 additional CSE-214-style problems with Java solutions

How these problems were derived

The five uploaded CSE-214 problems emphasize runtime policy changes, state-dependent operations, centralized coordination, fixed workflows with variable steps, and interchangeable decision rules. This practice set keeps that same exam-oriented structure while adding fresh scenarios for Strategy, State, Mediator, Observer, Command, and Template Method.

Pattern coverage

Pattern	Problems	Core cue
Strategy	1-2	Swap an algorithm/policy at runtime
State	3-4	Same operation changes behavior by current state
Mediator	5-6	Objects coordinate through a central hub
Observer	7-8, 14	One subject broadcasts changes to subscribers
Command	9-10, 13	Encapsulate an action as an object
Template Method	11-12	Fix the algorithm skeleton; vary selected steps

Exam tip: First identify what changes. If the answer is “the algorithm/policy,” think Strategy; “the object status,” State; “who talks to whom,” Mediator; “who should be notified,” Observer; “an action that should be queued/undone,” Command; “the sequence stays fixed but steps vary,” Template Method.

Problem 1: Dynamic Route Pricing

STRATEGY

Problem statement

A ride-sharing application calculates the final fare for a trip. The system supports three pricing policies: Standard, Peak Hour, and Student. The rider selects the preferred policy, but the policy may be changed while the application is running. Each policy receives the base fare and returns the final fare. The RideFareCalculator should not contain a large if/else chain for the pricing rules. Required operations: setPricingStrategy(...), calculateFare(...).

Pattern identification

The pricing rule can be swapped at runtime: Standard, Peak-Hour, and Student pricing all compute a fare differently behind one common interface.

Java solution

```
interface PricingStrategy {
    double calculate(double baseFare, double distanceKm);
}

class StandardPricing implements PricingStrategy {
    public double calculate(double baseFare, double d) {
        return baseFare + d * 12;
    }
}

class PeakPricing implements PricingStrategy {
    public double calculate(double baseFare, double d) {
        return 1.5 * (baseFare + d * 12);
    }
}

class StudentPricing implements PricingStrategy {
    public double calculate(double baseFare, double d) {
        return 0.8 * (baseFare + d * 12);
    }
}

class RideFareCalculator {
    private PricingStrategy strategy;

    RideFareCalculator(PricingStrategy strategy) {
        this.strategy = strategy;
    }

    void setPricingStrategy(PricingStrategy strategy) {
        this.strategy = strategy;
    }

    double calculateFare(double baseFare, double distanceKm) {
        return strategy.calculate(baseFare, distanceKm);
    }
}
```

Useful extension / exam variation: Add a CorporatePricing class without modifying RideFareCalculator.

What to say in the exam

The pricing rule can be swapped at runtime: Standard, Peak-Hour, and Student pricing all compute a fare differently behind one common interface. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 2: File Compression Service

STRATEGY

Problem statement

A backup program can compress a file using ZIP, GZIP, or BZIP2. The user chooses a compression method at startup and may change it before another backup. The BackupService should expose setCompressionStrategy(...) and createBackup(...). Do not put the compression-specific logic inside BackupService.

Pattern identification

ZIP, GZIP, and BZIP2 are interchangeable algorithms. The service chooses one algorithm without knowing its internal implementation.

Java solution

```
interface CompressionStrategy {
    byte[] compress(byte[] data);
}

class ZipCompression implements CompressionStrategy {
    public byte[] compress(byte[] data) {
        System.out.println("Compressing with ZIP");
        return data; // simplified
    }
}

class GzipCompression implements CompressionStrategy {
    public byte[] compress(byte[] data) {
        System.out.println("Compressing with GZIP");
        return data; // simplified
    }
}

class BackupService {
    private CompressionStrategy strategy;

    BackupService(CompressionStrategy strategy) {
        this.strategy = strategy;
    }

    void setCompressionStrategy(CompressionStrategy strategy) {
        this.strategy = strategy;
    }

    byte[] createBackup(byte[] data) {
        return strategy.compress(data);
    }
}
```

Useful extension / exam variation: Let the user select the strategy from a Map registry.

What to say in the exam

ZIP, GZIP, and BZIP2 are interchangeable algorithms. The service chooses one algorithm without knowing its internal implementation. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 3: Vending Machine Controller

STATE

Problem statement

A vending machine has four states: IDLE, HAS_COIN, DISPENSING, and OUT_OF_STOCK. Operations are insertCoin(), selectItem(), and dispense(). Invalid operations should print a message and leave the state unchanged. After an item is dispensed, the machine returns to IDLE. If stock becomes zero, the machine enters OUT_OF_STOCK.

Pattern identification

The same operation behaves differently depending on the machine state. State objects encapsulate those transitions instead of a large conditional block.

Java solution

```
interface VendingState {
    void insertCoin(VendingMachine m);
    void selectItem(VendingMachine m);
    void dispense(VendingMachine m);
}

class VendingMachine {
    VendingState state = new IdleState();
    int stock;
    boolean hasCoin;

    VendingMachine(int stock) { this.stock = stock; }
    void setState(VendingState s) { state = s; }
    void insertCoin() { state.insertCoin(this); }
    void selectItem() { state.selectItem(this); }
    void dispense() { state.dispense(this); }
}

class IdleState implements VendingState {
    public void insertCoin(VendingMachine m) { m.hasCoin = true; m.setState(new HasCoinState()); }
    public void selectItem(VendingMachine m) { System.out.println("Insert coin first"); }
    public void dispense(VendingMachine m) { System.out.println("Select an item first"); }
}

class HasCoinState implements VendingState {
    public void insertCoin(VendingMachine m) { System.out.println("Coin already inserted"); }
    public void selectItem(VendingMachine m) { m.setState(new DispensingState()); }
    public void dispense(VendingMachine m) { System.out.println("Select item first"); }
}

class DispensingState implements VendingState {
    public void insertCoin(VendingMachine m) { System.out.println("Please wait"); }
    public void selectItem(VendingMachine m) { System.out.println("Already dispensing"); }
    public void dispense(VendingMachine m) {
        if (m.stock <= 0) { m.setState(new OutOfStockState()); return; }
        m.stock--;
        m.hasCoin = false;
        System.out.println("Item dispensed");
        m.setState(m.stock == 0 ? new OutOfStockState() : new IdleState());
    }
}

class OutOfStockState implements VendingState {
    public void insertCoin(VendingMachine m) { System.out.println("Out of stock"); }
    public void selectItem(VendingMachine m) { System.out.println("Out of stock"); }
    public void dispense(VendingMachine m) { System.out.println("Out of stock"); }
}
```

Useful extension / exam variation: Add a MaintenanceState where only refill() is valid.

What to say in the exam

The same operation behaves differently depending on the machine state. State objects encapsulate those transitions instead of a large conditional block. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 4: Media Player Playback

STATE

Problem statement

Design a media player with states STOPPED, PLAYING, and PAUSED. The operations are play(), pause(), and stop(). In STOPPED, play starts playback; in PLAYING, pause pauses playback; in PAUSED, play resumes. Invalid operations should display an appropriate message. The Player object must remain the same while its state changes.

Pattern identification

Play, pause, and stop have different effects in each playback state. State localizes those rules and transitions.

Java solution

```
interface PlayerState {
    void play(Player p);
    void pause(Player p);
    void stop(Player p);
}

class Player {
    PlayerState state = new StoppedState();
    void setState(PlayerState s) { state = s; }
    void play() { state.play(this); }
    void pause() { state.pause(this); }
    void stop() { state.stop(this); }
}

class StoppedState implements PlayerState {
    public void play(Player p) { System.out.println("Playing"); p.setState(new PlayingState()); }
    public void pause(Player p) { System.out.println("Cannot pause: stopped"); }
    public void stop(Player p) { System.out.println("Already stopped"); }
}

class PlayingState implements PlayerState {
    public void play(Player p) { System.out.println("Already playing"); }
    public void pause(Player p) { System.out.println("Paused"); p.setState(new PausedState()); }
    public void stop(Player p) { System.out.println("Stopped"); p.setState(new StoppedState()); }
}

class PausedState implements PlayerState {
    public void play(Player p) { System.out.println("Resumed"); p.setState(new PlayingState()); }
    public void pause(Player p) { System.out.println("Already paused"); }
    public void stop(Player p) { System.out.println("Stopped"); p.setState(new StoppedState()); }
}
```

Useful extension / exam variation: Add BUFFERING and ERROR states without adding conditionals to Player.

What to say in the exam

Play, pause, and stop have different effects in each playback state. State localizes those rules and transitions. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 5: Air Traffic Control Coordinator

MEDIATOR

Problem statement

An airport has Aircraft objects and one ControlTower. Aircraft may request landing, request takeoff, and report that a runway has been released. Aircraft must not hold direct references to other aircraft. The tower decides who can use the runway and informs waiting aircraft.

Pattern identification

Planes must not communicate directly with one another. The control tower is the central mediator that coordinates runway access and notifications.

Java solution

```
interface AirMediator {
    void requestLanding(Aircraft a);
    void requestTakeoff(Aircraft a);
    void runwayReleased();
}

class ControlTower implements AirMediator {
    private Aircraft runwayUser;

    public void requestLanding(Aircraft a) {
        if (runwayUser == null) {
            runwayUser = a;
            a.notifyMessage("Landing approved");
        } else a.notifyMessage("Landing queued");
    }

    public void requestTakeoff(Aircraft a) {
        if (runwayUser == null) {
            runwayUser = a;
            a.notifyMessage("Takeoff approved");
        } else a.notifyMessage("Takeoff queued");
    }

    public void runwayReleased() {
        runwayUser = null;
        System.out.println("Runway available");
    }
}

class Aircraft {
    private final AirMediator mediator;
    private final String id;

    Aircraft(String id, AirMediator mediator) {
        this.id = id; this.mediator = mediator;
    }

    void land() { mediator.requestLanding(this); }
    void takeOff() { mediator.requestTakeoff(this); }
    void notifyMessage(String msg) { System.out.println(id + ": " + msg); }
}
```

Useful extension / exam variation: Add a GroundControl participant without changing Aircraft-to-Aircraft relationships.

What to say in the exam

Planes must not communicate directly with one another. The control tower is the central mediator that coordinates runway access and notifications. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 6: Smart Home Scene Coordinator

MEDIATOR

Problem statement

A smart home contains Light, AirConditioner, Curtain, and Alarm devices. When the user activates the "Movie Mode" scene, one central HomeHub sends commands to all devices. When a device changes state, it reports to the hub. Devices should not directly invoke one another. Implement activateMovieMode() and a minimal device registration mechanism.

Pattern identification

Lights, AC, curtains, and alarm communicate through a HomeHub instead of maintaining many-to-many references.

Java solution

```
interface HomeMediator {
    void deviceChanged(Device d, String event);
    void activateMovieMode();
}

interface Device {
    void receive(String command);
    String name();
}

class HomeHub implements HomeMediator {
    private final java.util.List<Device> devices = new java.util.ArrayList<>();

    void register(Device d) { devices.add(d); }

    public void deviceChanged(Device d, String event) {
        System.out.println(d.name() + " -> " + event);
    }

    public void activateMovieMode() {
        for (Device d : devices) d.receive("MOVIE_MODE");
    }
}

class Light implements Device {
    public void receive(String c) { System.out.println("Lights: " + c); }
    public String name() { return "Light"; }
}

class AirConditioner implements Device {
    public void receive(String c) { System.out.println("AC: " + c); }
    public String name() { return "AC"; }
}
```

Useful extension / exam variation: Make commands depend on time of day while keeping device classes unaware of one another.

What to say in the exam

Lights, AC, curtains, and alarm communicate through a HomeHub instead of maintaining many-to-many references. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 7: Stock Price Alert System

OBSERVER

Problem statement

A trading dashboard tracks a stock price. Multiple observers - EmailAlert, MobileAlert, and DashboardDisplay - subscribe to the stock. Whenever the price changes, all registered observers are notified. Provide subscribe(), unsubscribe(), and setPrice().

Pattern identification

Many subscribers need automatic notifications when one subject changes. The stock object should not know the internal details of each subscriber.

Java solution

```
interface Observer {
    void update(String symbol, double price);
}

class Stock {
    private final String symbol;
    private double price;
    private final java.util.List<Observer> observers = new java.util.ArrayList<>();

    Stock(String symbol) { this.symbol = symbol; }
    void subscribe(Observer o) { observers.add(o); }
    void unsubscribe(Observer o) { observers.remove(o); }

    void setPrice(double price) {
        this.price = price;
        for (Observer o : observers) o.update(symbol, price);
    }
}

class EmailAlert implements Observer {
    public void update(String s, double p) { System.out.println("Email: " + s + " = " + p); }
}

class DashboardDisplay implements Observer {
    public void update(String s, double p) { System.out.println("Dashboard: " + s + " = " + p); }
}
```

Useful extension / exam variation: Notify only when the absolute price change is at least 2.0.

What to say in the exam

Many subscribers need automatic notifications when one subject changes. The stock object should not know the internal details of each subscriber. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 8: Weather Station Notifications

OBSERVER

Problem statement

A weather station measures temperature and humidity. PhoneDisplay, WebDisplay, and HeatWarningService subscribe to the station. Whenever a new measurement arrives, all observers are notified. HeatWarningService should print a warning only when temperature is at least 38 C, but the station must still notify it on every update.

Pattern identification

The weather station is the subject, while phone display, web display, and warning service are independent observers.

Java solution

```
interface WeatherObserver {
    void update(double temperature, double humidity);
}

class WeatherStation {
    private final java.util.List<WeatherObserver> observers = new java.util.ArrayList<>();
    void addObserver(WeatherObserver o) { observers.add(o); }
    void removeObserver(WeatherObserver o) { observers.remove(o); }

    void setMeasurement(double t, double h) {
        for (WeatherObserver o : observers) o.update(t, h);
    }
}

class PhoneDisplay implements WeatherObserver {
    public void update(double t, double h) { System.out.println("Phone: " + t + " C, " + h
+ "%"); }
}

class HeatWarningService implements WeatherObserver {
    public void update(double t, double h) {
        if (t >= 38) System.out.println("WARNING: extreme heat");
    }
}
```

Useful extension / exam variation: Add a LoggerObserver that records the latest five measurements.

What to say in the exam

The weather station is the subject, while phone display, web display, and warning service are independent observers. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 9: Smart Remote with Undo

COMMAND

Problem statement

A smart home remote controls a Light and a Fan. Buttons are configured with commands: LightOnCommand, LightOffCommand, FanStartCommand, and FanStopCommand. The remote calls execute() without knowing how each device works. Add an undo() operation so the last command can be reversed.

Pattern identification

The remote should issue different requests through one uniform execute() interface, and the same abstraction enables undo.

Java solution

```
interface Command {
    void execute();
    void undo();
}

class Light {
    void on() { System.out.println("Light ON"); }
    void off() { System.out.println("Light OFF"); }
}

class LightOnCommand implements Command {
    private final Light light;
    LightOnCommand(Light l) { light = l; }
    public void execute() { light.on(); }
    public void undo() { light.off(); }
}

class Remote {
    private Command last;
    void press(Command c) { last = c; c.execute(); }
    void undo() { if (last != null) last.undo(); }
}
```

Useful extension / exam variation: Add MacroCommand that executes a list of commands in sequence and undoes them in reverse order.

What to say in the exam

The remote should issue different requests through one uniform execute() interface, and the same abstraction enables undo. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 10: Document Editor Actions

COMMAND

Problem statement

A document editor has InsertTextCommand, DeleteTextCommand, and SaveCommand. Toolbar buttons should execute commands, while an ActionHistory stores executed commands. Implement execute(), undo(), and redo() for one or more commands. For a 30-minute exam solution, focus on the command interface, two concrete commands, and history management.

Pattern identification

Insert, delete, and format actions become command objects that can be queued, logged, and undone without coupling the editor to a UI button.

Java solution

```
interface EditorCommand {
    void execute();
    void undo();
}

class Editor {
    private String text = "";
    void insert(String s) { text += s; }
    void deleteLast(int n) { text = text.substring(0, text.length() - n); }
    String getText() { return text; }
}

class InsertTextCommand implements EditorCommand {
    private final Editor editor; private final String text;
    InsertTextCommand(Editor e, String t) { editor=e; text=t; }
    public void execute() { editor.insert(text); }
    public void undo() { editor.deleteLast(text.length()); }
}

class ActionHistory {
    private final java.util.Deque<EditorCommand> done = new java.util.ArrayDeque<>();
    void execute(EditorCommand c) { c.execute(); done.push(c); }
    void undo() { if (!done.isEmpty()) done.pop().undo(); }
}
```

Useful extension / exam variation: Add redo using a second stack and clear redo after a new execute().

What to say in the exam

Insert, delete, and format actions become command objects that can be queued, logged, and undone without coupling the editor to a UI button. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 11: Daily Report Generator

TEMPLATE METHOD

Problem statement

A company creates reports from CSV, JSON, and XML sources. The overall workflow must always be: loadData -> validate -> transform -> format -> publish. CSVReport and JSONReport implement the source-specific steps. Avoid copying the full workflow into every subclass.

Pattern identification

All report generators follow the same fixed sequence, while subclasses customize individual steps.

Java solution

```
abstract class ReportGenerator {
    public final void generate() {
        loadData();
        validate();
        transform();
        format();
        publish();
    }

    protected abstract void loadData();
    protected abstract void transform();
    protected abstract void format();

    protected void validate() { System.out.println("Common validation"); }
    protected void publish() { System.out.println("Report published"); }
}

class CSVReport extends ReportGenerator {
    protected void loadData() { System.out.println("Read CSV"); }
    protected void transform() { System.out.println("Transform CSV data"); }
    protected void format() { System.out.println("Format as table"); }
}

class JSONReport extends ReportGenerator {
    protected void loadData() { System.out.println("Read JSON"); }
    protected void transform() { System.out.println("Transform JSON data"); }
    protected void format() { System.out.println("Format as JSON report"); }
}
```

Useful extension / exam variation: Make validate() a hook: subclasses may override it, but the order of the template method must not change.

What to say in the exam

All report generators follow the same fixed sequence, while subclasses customize individual steps. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 12: Online Payment Processing Pipeline

TEMPLATE METHOD

Problem statement

An online shop supports CardPayment, MobileWalletPayment, and BankTransferPayment. Every payment must execute this sequence: validateCustomer -> validatePaymentData -> authorize -> charge -> generateReceipt. The sequence is fixed and must not be duplicated in every subclass. A failed validation or authorization stops the process.

Pattern identification

Card, mobile-wallet, and bank-transfer payments follow the same control sequence but customize validation, charging, and receipt generation.

Java solution

```
abstract class PaymentProcessor {
    public final boolean process() {
        if (!validateCustomer()) return false;
        if (!validatePaymentData()) return false;
        if (!authorize()) return false;
        charge();
        generateReceipt();
        return true;
    }

    protected boolean validateCustomer() { return true; }
    protected abstract boolean validatePaymentData();
    protected abstract boolean authorize();
    protected abstract void charge();
    protected void generateReceipt() { System.out.println("Receipt generated"); }
}

class CardPayment extends PaymentProcessor {
    protected boolean validatePaymentData() { return true; }
    protected boolean authorize() { System.out.println("Card authorized"); return true; }
    protected void charge() { System.out.println("Card charged"); }
}

class MobileWalletPayment extends PaymentProcessor {
    protected boolean validatePaymentData() { return true; }
    protected boolean authorize() { System.out.println("Wallet OTP verified"); return true; }
    protected void charge() { System.out.println("Wallet debited"); }
}
```

Useful extension / exam variation: Add a fraudCheck() hook that runs only when a subclass enables it.

What to say in the exam

Card, mobile-wallet, and bank-transfer payments follow the same control sequence but customize validation, charging, and receipt generation. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 13: Restaurant Order Queue

COMMAND

Problem statement

A restaurant POS accepts commands such as PreparePizza, PrepareBurger, and CancelOrder. A KitchenQueue stores commands and executes them later. The POS should not contain food-preparation logic. Implement addCommand() and processAll().

Pattern identification

Kitchen actions can be represented as objects and queued for later execution. The waiter and kitchen remain decoupled.

Java solution

```
interface KitchenCommand { void execute(); }

class Kitchen {
    void preparePizza() { System.out.println("Pizza prepared"); }
    void prepareBurger() { System.out.println("Burger prepared"); }
}

class PreparePizza implements KitchenCommand {
    private final Kitchen kitchen;
    PreparePizza(Kitchen k) { kitchen = k; }
    public void execute() { kitchen.preparePizza(); }
}

class KitchenQueue {
    private final java.util.Queue<KitchenCommand> q = new java.util.ArrayDeque<>();
    void addCommand(KitchenCommand c) { q.add(c); }
    void processAll() { while (!q.isEmpty()) q.remove().execute(); }
}
```

Useful extension / exam variation: Support priority queues so urgent commands are executed first.

What to say in the exam

Kitchen actions can be represented as objects and queued for later execution. The waiter and kitchen remain decoupled. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Problem 14: Delivery Notification Hub

OBSERVER

Problem statement

A `DeliveryOrder` has statuses `PACKED`, `SHIPPED`, `OUT_FOR_DELIVERY`, and `DELIVERED`. Whenever the status changes, `CustomerApp`, `EmailService`, and `SMSService` must be notified. The order should not know how each service sends its notification. Implement `attach()`, `detach()`, and `changeStatus()`.

Pattern identification

A delivery status change should fan out to multiple independent subscribers such as customer app, email, and SMS.

Java solution

```
interface DeliveryObserver {
    void update(String orderId, String status);
}

class DeliveryOrder {
    private final String id;
    private String status;
    private final java.util.List<DeliveryObserver> observers = new java.util.ArrayList<>();

    DeliveryOrder(String id) { this.id = id; }
    void attach(DeliveryObserver o) { observers.add(o); }
    void detach(DeliveryObserver o) { observers.remove(o); }

    void changeStatus(String s) {
        status = s;
        for (DeliveryObserver o : observers) o.update(id, status);
    }
}

class CustomerApp implements DeliveryObserver {
    public void update(String id, String status) { System.out.println("App: " + id + " -> "
+ status); }
}

class SMSService implements DeliveryObserver {
    public void update(String id, String status) { System.out.println("SMS: " + id + " -> "
+ status); }
}
```

Useful extension / exam variation: Notify some observers only for selected statuses by adding filtering inside the observer, not inside `DeliveryOrder`.

What to say in the exam

A delivery status change should fan out to multiple independent subscribers such as customer app, email, and SMS. The key is to keep the context class small and delegate the variable behaviour to the appropriate behavioural object(s).

Quick Recognition Sheet

Strategy - Context owns one strategy object and delegates the operation. The strategy can be changed at runtime.

State - Context delegates to a state object. Calls can trigger state transitions, so behaviour depends on current state.

Mediator - Colleagues communicate through one mediator instead of many direct references.

Observer - Subject keeps subscribers and pushes a change notification to them.

Command - A request is wrapped as an object, enabling queueing, logging, undo, macro commands, or delayed execution.

Template Method - An abstract class fixes the order of the algorithm in a final template method; subclasses customize steps/hooks.

Common CSE-214 implementation traps

- Do not put the entire pattern into one giant class with if/else branches. The purpose is to move the varying behaviour into collaborating objects.
- For State and Strategy, the Context usually owns a reference to an interface type, not to every concrete implementation.
- For Mediator, avoid direct references between colleagues when the problem explicitly says communication is centralized.
- For Observer, notify current subscribers after a state/data change and provide attach/detach operations when required.
- For Template Method, make the top-level workflow final so subclasses cannot reorder the required steps.
- For Command, put the receiver call inside the command object. The invoker should only know Command.

Practice method

Try each problem once from the statement alone. On the second pass, draw the minimum class diagram: Context/Subject/Invoker, one behavioural interface, and two or three concrete classes. Then code only the operations the statement explicitly requires.